Intermediate-Code Automorphisms in Pieceable Fault-Tolerant Quantum Computation

M.Subhi Abo Rdan*
Department of Physics, Massachusetts Institute of Technology
(Dated: May 2026)

Pieceable fault tolerance implements non-transversal logical gates by decomposing them into circuit pieces separated by intermediate error-correction steps. During this evolution, the encoded code space changes dynamically, producing intermediate codes with symmetry structures distinct from those of the original stabilizer code.

In this work, we study automorphisms of these dynamically evolving intermediate codes in pieceable fault-tolerant constructions. Focusing first on logical $\overline{CZ}$ constructions for the $[[5,1,3]]$ code, where all intermediate codes remain stabilizer codes, we perform automorphism searches directly on the intermediate stabilizer representations. These searches produce fused multi-qubit logical Clifford constructions that naturally combine logical operations with the entangling structure already present in the pieceable circuit.

We then investigate logical $\overline{CCZ}$ constructions, where the intermediate codes become non-stabilizer due to the presence of non-Clifford gates. To study symmetries in this regime, we extend the analysis to equivalent Codeword Stabilized (CWS) representations of the evolving codes. In particular, we show that the classical binary code associated with the embedded CWS representation remains invariant throughout the intermediate CCZ evolution, while the non-Clifford dynamics are transferred into the evolving hypergraph structure.

Our results illustrate both the potential and the limitations of permutation-based symmetry methods for fault-tolerant logical gate synthesis. More broadly, they suggest that pieceable fault-tolerant circuits may naturally be viewed as dynamically evolving families of quantum codes whose intermediate symmetry structures can be exploited for logical circuit optimization.

I. INTRODUCTION

Quantum error correction provides a framework for protecting quantum information against decoherence, imperfect control operations, and measurement noise. By encoding logical qubits into larger entangled states of many physical qubits, quantum error-correcting codes allow reliable logical computation even in the presence of physical faults. A central challenge in fault-tolerant quantum computation is constructing logical gate implementations that both preserve the encoded information and limit the propagation of physical errors throughout the circuit [1].

The Eastin–Knill theorem implies that no quantum error-correcting code can realize a universal set of transversal logical gates [2]. Furthermore, the Bravyi–König theorem constrains the set of logical gates implementable by constant-depth local circuits in topological stabilizer codes [3]. As a result, fault-tolerant implementations of non-Clifford gates typically require costly techniques such as magic-state distillation, lattice surgery, or code switching, all of which introduce significant resource overheads [4, 12, 13].

Pieceable fault tolerance provides an alternative framework in which non-transversal logical gates can be implemented fault tolerantly by interleaving circuit pieces with rounds of intermediate error correction. Previous constructions demonstrated how to realize fault-tolerant C^nZ gates for nondegenerate stabilizer codes possessing a complete set of fault-tolerant single-qubit Clifford gates [5].

Constructing optimized logical gate implementations remains a difficult problem due to the large search space of stabilizer code automorphisms. Restricting the search to permutation automorphisms yields logical operations generated primarily by local Clifford and SWAP gates, significantly limiting the class of realizable entangling constructions. More general automorphism searches rapidly become computationally intractable due to the exponential growth of the Clifford search space [8, 9].

Existing automorphism-based approaches, such as `AutQEC` and related stabilizer-code symmetry frameworks, primarily study automorphisms of static stabilizer-code representations. These methods are highly effective for identifying logical Clifford operations generated by local Clifford transformations and coordinate permutations. However, their ability to generate nontrivial entangling logical constructions remains limited. In particular, in our experiments these methods fail to directly recover the pieceable fault-tolerant logical $\overline{CZ}$ construction between two blocks of the $[[5,1,3]]$ code using permutation automorphisms alone [5, 7].

Embedding-based extensions can partially overcome this restriction by translating permutation symmetries of enlarged auxiliary codes into entangling logical operations, but the resulting searches rapidly become computationally expensive and difficult to scale [7]. Moreover, such approaches generally do not capture the dynamically evolving structure of intermediate codes aris-

* This work was completed as part of the final project for MIT 8.371: Quantum Information Science II; (msubhi_a@mit.edu)

ing during non-transversal fault-tolerant logical gate constructions.

In this work, we investigate dynamically evolving code symmetries arising during pieceable fault-tolerant logical gate constructions. Rather than studying only the automorphisms of a static stabilizer code, we analyze the sequence of intermediate codes generated throughout the evolution of the fault-tolerant circuit. Since the intermediate codes appearing during pieceable logical $\overline{CZ}$ constructions remain stabilizer codes, automorphism searches may be performed directly on the evolving intermediate stabilizer representations. This viewpoint naturally treats pieceable fault-tolerant circuits as dynamically evolving families of quantum codes possessing distinct symmetry structures at different stages of the circuit evolution.

By combining intermediate-code automorphism searches with pieceable constructions on the $[[5,1,3]]$ code, we obtain fused multi-qubit logical Clifford gate constructions that naturally combine logical operations with the entangling structure already present in the circuit. In contrast to independently synthesizing logical Clifford layers together with logical entangling gates, the resulting constructions exploit symmetries of the intermediate codes themselves to generate optimized logical gate primitives. In particular, we observe that the automorphism groups of the evolving intermediate codes shrink rapidly as the round-robin entangling structure becomes increasingly complex.

We further investigate pieceable fault-tolerant $\overline{CCZ}$ constructions, where the intermediate codes are generally no longer stabilizer codes due to the presence of non-Clifford gates. To study symmetries in this regime, we extend the analysis to equivalent Codeword Stabilized (CWS) representations of the evolving intermediate codes. We show that the diagonal structure of CCZ gates preserves the associated classical binary code of the embedded CWS representation, while the non-Clifford dynamics are transferred into the evolving graph and hypergraph structure. This naturally separates the intermediate evolution into two distinct components: a static classical-code sector and a dynamically evolving hypergraph sector encoding the non-Clifford structure. Our results illustrate both the potential and the practical limitations of permutation-based symmetry methods in the non-stabilizer setting, where hypergraph-preservation constraints appear to strongly suppress nontrivial automorphisms.

The remainder of this paper is organized as follows. Section II A, Section II B, and Section II C review the stabilizer formalism, fault-tolerant quantum computation, and pieceable fault tolerance. Section II D and Section II E review the symplectic and CWS representations used throughout this work. Section III A discusses stabilizer-code automorphisms and existing automorphism-based logical gate constructions. Section IV studies automorphisms of dynamically evolving intermediate stabilizer codes arising during pieceable logical $\overline{CZ}$ constructions on the [[5,1,3]] code. Finally, Section V investigates non-stabilizer intermediate codes arising during pieceable $\overline{CCZ}$ constructions through equivalent CWS and hypergraph representations.

II. BACKGROUND

A. Stabilizer Formalism

a. ***Stabilizer codes.*** An $[[n,k,d]]$ quantum error-correcting code is a 2^k-dimensional subspace $\mathcal{C}$ of the 2^n-dimensional Hilbert space $\mathcal{H}_2^{\otimes n}$. It encodes k logical qubits into n physical qubits. Stabilizer codes are specified by an abelian subgroup $S \subset \mathcal{P}_n$ of the n-qubit Pauli group, where $-I \notin S$. The code space is defined as the simultaneous $+1$ eigenspace of all stabilizer generators:

$$\mathcal{C}_S = \left\{ |\psi\rangle \in \mathcal{H}_2^{\otimes n} \ : \ g\,|\psi\rangle = +\,|\psi\rangle \quad \forall g \in S \right\}.$$

Each independent stabilizer generator halves the dimension of the Hilbert space. Therefore, an $[[n,k,d]]$ stabilizer code has $n-k$ independent stabilizer generators, leaving a 2^k-dimensional code space.

The encoded basis states are

$$|\overline{x_1 x_2 \cdots x_k}\rangle\,, \qquad x_i \in \{0,1\},$$

corresponding to the 2^k computational basis states of the k encoded logical qubits. The overline indicates that these logical states are encoded into the full n-qubit physical system.

b. ***Logical operators.*** Logical Pauli operators are Pauli operators that preserve the code space while acting nontrivially on the encoded logical qubits. Equivalently, they are elements of the normalizer $N(S)$ that are not themselves stabilizers:

$$\overline{P} \in N(S) \setminus S.$$

Here, $N(S)$ is the set of Pauli operators that commute with every stabilizer element. Operators in S act trivially on the code space, while representatives of $N(S) \setminus S$ implement logical Pauli operators such as $\overline{X}_i$ and $\overline{Z}_i$.

c. ***Error model.*** Any quantum error channel may be expanded in the Pauli operator basis, so it suffices to analyze the effect of Pauli errors. A single-qubit Pauli error can take a state out of the code space by flipping the eigenvalue of any stabilizer generator with which it anticommutes. For example, an X error on a given qubit anticommutes with stabilizers containing a Z or Y on that qubit, while a Z error anticommutes with stabilizers containing an X or Y on that qubit. Measuring the stabilizer generators therefore produces a syndrome that identifies which stabilizers have changed sign. If distinct correctable errors produce distinct syndromes, then the corresponding Pauli correction can be applied to return the state to the code space.

The distance d of a stabilizer code is the minimum weight of a nontrivial logical Pauli operator:

$$d = \min_{\overline{P} \in N(S) \setminus S} \mathrm{wt}(\overline{P}).$$

A code of distance d can detect up to $d-1$ arbitrary Pauli errors and correct up to $\lfloor (d-1)/2 \rfloor$ arbitrary errors [1].

d. ***Example: the*** $[[5,1,3]]$ ***code.*** A central example throughout this work is the five-qubit perfect code, an $[[5,1,3]]$ stabilizer code. It is the smallest possible quantum code capable of correcting an arbitrary single-qubit error. One standard choice of stabilizer generators is

$$S = \left\langle \begin{matrix} XZZXI \\ IXZZX \\ XIXZZ \\ ZXIXZ \end{matrix} \right\rangle.$$

This code encodes one logical qubit into five physical qubits and has distance three. A convenient choice of logical Pauli operators is

$$\overline{X} = XXXXX, \qquad \overline{Z} = ZZZZZ.$$

Although the above logical representatives have weight five, equivalent minimum-weight representatives may be obtained by multiplying by stabilizer elements. One possible choice is

$$\overline{X}_{\min} = -XIZIX \qquad \overline{Z}_{\min} = -YIXIY.$$

The minimum weight of a nontrivial logical operator is therefore three, implying that the code distance is $d = 3$.

B. Fault-Tolerant Quantum Computation

a. ***Fault tolerance and error correction.*** The goal of fault-tolerant quantum computation is to control the propagation and accumulation of physical errors throughout a quantum circuit while operating on encoded logical states. Logical operations on encoded qubits are therefore implemented through fault-tolerant procedures designed to prevent a small number of physical faults from spreading into uncorrectable logical errors.

For a distance-d quantum code, up to $t = \left\lfloor \frac{d-1}{2} \right\rfloor$ physical errors may be corrected within each encoded block. A fault-tolerant logical operation must therefore ensure that a sufficiently small number of faults during the implementation produces at most t errors per code block, allowing subsequent rounds of error correction to recover the encoded state.

Errors in a fault-tolerant circuit may arise from several sources, including residual errors propagated from previous circuit locations, imperfect physical gate implementations, state preparation, and noisy measurements. Various fault-tolerant syndrome extraction procedures, such as Shor-style and Steane-style error correction, have been developed to suppress correlated error propagation during stabilizer measurements.

b. ***Error propagation and the Clifford group.*** A useful model for analyzing error propagation is to represent a faulty gate as an ideal unitary gate U followed by an error operator E. The resulting operation may be written as

$$EU = U(U^\dagger E U).$$

Thus, under conjugation by the ideal gate, the effective propagated error becomes

$$E' = U^\dagger E U.$$

If the propagated error remains within the assumed correctable error model, then the stabilizer code may continue to correct the resulting faults.

An especially important class of operations satisfying this property is the Clifford group. The Clifford group on n qubits is defined as the set of unitary operators that preserve the Pauli group under conjugation:

$$\mathcal{C}_n = \left\{ U \;\middle|\; U \mathcal{P}_n U^\dagger = \mathcal{P}_n \right\}.$$

Equivalently, Clifford operators map Pauli operators to Pauli operators under conjugation. The Clifford group is generated by the Hadamard gate H, phase gate S, and two-qubit entangling gates such as $CNOT$ or CZ.

Because Clifford operations preserve Pauli errors, they play a central role in fault-tolerant quantum computation and stabilizer error correction. However, the Clifford group alone is not computationally universal, since circuits composed entirely of Clifford gates may be efficiently simulated classically via the Gottesman–Knill theorem[11]. Universal quantum computation, therefore, requires the addition of at least one non-Clifford operation, such as the T gate, Toffoli gate, or CCZ gate[4]..

c. ***Transversal logical operators.*** In addition to preserving the error model, fault-tolerant logical gates must also limit the spatial propagation of errors across encoded blocks. A particularly important property in this context is transversality. A transversal logical gate acts independently across corresponding physical qubits of different code blocks and therefore prevents a single physical fault from spreading to multiple qubits within the same block.

Many stabilizer codes admit transversal implementations of some Clifford gates, and certain CSS codes additionally admit transversal non-Clifford gates. For example, the $[[15,1,3]]$ Reed–Muller code admits a transversal logical T gate. Nevertheless, the Eastin–Knill theorem proves that no finite-dimensional quantum error-correcting code can realize a universal set of transversal logical gates [2]. More generally, the Bravyi–König theorem places additional locality constraints on constant-depth logical gate implementations in topological stabilizer codes [3]. These no-go theorems motivate alternative fault-tolerant approaches for implementing universal logical gate sets.

C. Pieceable Fault Tolerance

Pieceable fault tolerance is a method for implementing non-transversal logical gates by decomposing them into several circuit pieces, with intermediate error correction inserted between pieces. The key idea is that although the full logical gate may not be transversal, each individual piece is chosen so that a small number of physical faults cannot spread to more than a correctable number of errors in any code block before the next error-correction step. We refer to the original construction for the full fault-tolerance analysis and for the details of the intermediate error-correction procedures [5].

The construction gives a general pieceable implementation of logical controlled-phase gates of the form $\overline{C^hZ}$, where C^hZ denotes the $(h+1)$-qubit controlled-Z gate.

At a high level, the construction consists of three stages. First, a prologue made of local Clifford gates maps the relevant logical Pauli operators $\overline{p_i}^{(j)}$ in each code block j into operators supported only on Z and I Paulis. Second, a round-robin circuit applies physical C^hZ gates between the corresponding supports of these transformed logical operators. Finally, an epilogue, given by the inverse of the prologue, maps the code back to its original representation.

For example, Fig. 1 shows the pieceable construction of a logical $\overline{CZ}$ gate on two 5-qubit code blocks. Since this construction uses only Clifford gates, the intermediate codes remain stabilizer codes. In contrast, Fig. 2 shows the round-robin construction for a logical $\overline{CCZ}$ gate between three code blocks. Because physical CCZ gates are non-Clifford, the intermediate codes obtained during this circuit are generally no longer stabilizer codes.

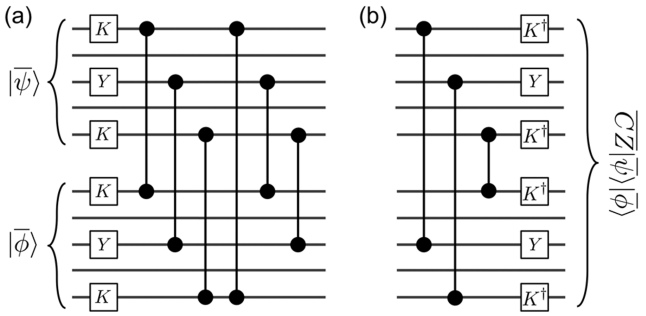

FIG. 1: Pieceable implementation of a logical $\overline{CZ}$ gate on two 5-qubit code blocks, adapted from Ref. [5]. The circuits C_a and C_b satisfy $\overline{CZ} = C_bC_a$. Here $K = SH$.

D. Symplectic Binary Representation and GF(4)

Stabilizer codes admit a correspondence with classical additive codes through the symplectic binary representation of Pauli operators [7, 9]. Any n-qubit Pauli operator can be represented by a binary vector of length $2n$ in

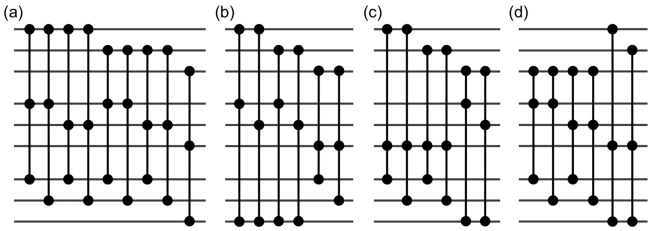

FIG. 2: Round-robin circuit used in the pieceable implementation of a logical $\overline{CCZ}$ gate between three code blocks, adapted from Ref. [5].

$(X|Z)$ form,

$$v = (a|b) \in \mathbb{F}_2^{2n},$$

where the binary vectors $a, b \in \mathbb{F}_2^n$ indicate the positions of X and Z components respectively. Under this representation,

$$I \mapsto (0|0), \qquad X \mapsto (1|0), \qquad Z \mapsto (0|1), \qquad Y \mapsto (1|1).$$

A stabilizer group generated by $n-k$ independent Pauli operators can therefore be represented by an $(n-k)\times 2n$ binary matrix whose rows are the symplectic representations of the generators. The row span of this matrix defines a classical additive code over $\mathbb{F}_2$. Commutation between Pauli operators is characterized by the symplectic inner product

$$\langle (a|b), (a'|b') \rangle_s = a \cdot b' + b \cdot a' \pmod 2.$$

Hence, stabilizer generators correspond to a self-orthogonal additive code under this symplectic form, which guarantees that the associated Pauli operators commute.

An equivalent formulation can be given over the finite field $\mathrm{GF}(4) = \{0, 1, \omega, \bar{\omega}\}$ [9]. The symplectic binary vector $v = (a|b)$ is mapped to a vector over $\mathrm{GF}(4)$ through $\phi(v) = a + \omega b$. Under this correspondence,

$$I \leftrightarrow 0, \qquad X \leftrightarrow 1, \qquad Z \leftrightarrow \omega, \qquad Y \leftrightarrow \bar{\omega}.$$

Additivity over $\mathrm{GF}(4)$ corresponds directly to the additive stabilizer structure. A stronger condition is linearity over $\mathrm{GF}(4)$, where multiplication by ω preserves the code space. Stabilizer codes arising from linear $\mathrm{GF}(4)$ codes are referred to as linear quantum codes. Such codes often possess larger automorphism groups since additional algebraic transformations preserve the code structure.

Some $\mathrm{GF}(4)$ codes additionally possess cyclic or constacyclic symmetries, which further enhance their automorphism groups [9]. For example, the $[[5, 1, 3]]$ 5-qubit code admits a cyclic $\mathrm{GF}(4)$ representation. These symmetries are particularly useful when studying logical gate implementations induced by code automorphisms.

Non-additive quantum codes generally correspond to non-stabilizer code constructions. One important example is the codeword stabilized (CWS) formalism discussed in Sec. II E.

As an example, consider the highly symmetric $[[5,1,3]]$ 5-qubit code with stabilizer generators

Generator	Binary $(X\|Z)$	GF(4)
$XZZXI$	$(10010\,\|\,01100)$	$(1,\omega,\omega,1,0)$
$IXZZX$	$(01001\,\|\,00110)$	$(0,1,\omega,\omega,1)$
$XIXZZ$	$(10100\,\|\,00011)$	$(1,0,1,\omega,\omega)$
$ZXIXZ$	$(01010\,\|\,10001)$	$(\omega,1,0,1,\omega)$

For the $[[5,1,3]]$ code, only two generators are needed in the GF(4)-linear description because each GF(4) generator accounts for a two-dimensional $\mathbb{F}_2$ span. The cyclic symmetry does not itself reduce the number of generators, but gives a convenient polynomial representation of the code [9].

E. Codeword Stabilized (CWS) Formalism

An important extension of the stabilizer formalism is the codeword stabilized (CWS) framework [10]. The relevant features for this work are summarized briefly below.

A CWS code is specified by two objects:

- an abelian subgroup S of the Pauli group, called the *word stabilizer*,
- a set of Pauli operators $\mathcal{W} = \{w_i\}$ called the *word operators*.

The stabilizer S uniquely defines a quantum state $|S\rangle$ stabilized by every element of S. The codewords of the CWS code are then generated as

$$|w_i\rangle = w_i|S\rangle.$$

Any CWS code admits a standard form locally equivalent to a graph-state stabilizer together with word operators consisting only of Z operators [10]. Consequently, a CWS code can be represented by a pair

$$(\mathcal{G},\mathcal{C}),$$

where $\mathcal{G}$ is a graph and $\mathcal{C}$ is a classical binary code whose binary strings specify the support of the Z components in the word operators.

An important fact is that every stabilizer code admits an equivalent description as a CWS code. This correspondence will later be used to study automorphisms and symmetries of intermediate non-stabilizer codes arising during pieceable fault-tolerant constructions.

III. CODE AUTOMORPHISMS AND LOGICAL GATES

A. Automorphisms of Stabilizer Codes

The full isomorphism group acting on additive GF(4) codes is generated by two types of transformations:

1. **Local GF(4) symmetries.** At each coordinate, the nonzero elements $\{1,\omega,\bar{\omega}\}$ may be permuted. These transformations form the symmetric group S_3 and are generated by multiplication by ω together with field conjugation. In the Pauli representation, these operations correspond to permutations of the Pauli operators X, Y, Z.
2. **Coordinate permutations.** The n coordinates may additionally be permuted according to

$$(c_1, c_2, \ldots, c_n) \to (c_{\pi(1)}, c_{\pi(2)}, \ldots, c_{\pi(n)}), \qquad \pi \in S_n.$$

Combining local S_3 actions on each coordinate with coordinate permutations gives the wreath product

$$G_n = S_3 \wr S_n,$$

whose order is $|G_n| = 6^n n!$. Only a subgroup of G_n preserves a given code. This subgroup is called the automorphism group of the code and is denoted $Aut(C) \subseteq G_n$. These automorphisms preserve the additive code structure and therefore preserve the corresponding stabilizer code [9].

As an example, the $[[5,1,3]]$ 5-qubit code corresponds to a $[5,2]_{\mathrm{GF}(4)}$ additive code. Although

$$|G_5| = 6^5 \cdot 5! = 933{,}120,$$

only 360 transformations preserve the code, giving $|Aut(C)| = 360$. This automorphism group is isomorphic to the alternating group $Aut(C) \cong A_6$, the group of even permutations on six elements [9].

Combining two copies of the $[[5,1,3]]$ code gives a $[[10,2,3]]$ code whose automorphism group contains independent automorphisms on each block together with permutations exchanging the two blocks:

$$Aut(C^{\oplus 2}) \cong (Aut(C) \times Aut(C)) \rtimes S_2.$$

Consequently, $|Aut(C^{\oplus 2})| = 360^2 \cdot 2 = 259200$.

B. Logical Clifford Gates from Permutation Automorphisms

A systematic method for finding automorphisms is obtained by mapping the GF(4) representation into the binary symplectic representation $[G_X \mid G_Z]$. Which then extended into the tripled binary representation

$$[G_X \mid G_Z \mid G_X \oplus G_Z],$$

where permutation automorphisms correspond to permutations of the columns preserving the row space [7].

Each allowed permutation corresponds to a physical Clifford transformation or qubit permutation. Three important classes are:

- **Hadamard transformations** generated by exchanging $(i, n+i)$, corresponding to the mapping $X \leftrightarrow Z$.

- **Phase transformations** generated by exchanging $(n+i, 2n+i)$, corresponding to the mapping $Z \leftrightarrow XZ$.
- **SWAP transformations** generated by simultaneously permuting $(i, j), (n+i, n+j), (2n+i, 2n+j)$.

These transformations generate physical circuits composed of single-qubit Clifford gates together with qubit permutations. Nontrivial permutations of the encoded operators therefore induce logical Clifford operations.

a. ***Circuit reconstruction from permutations.*** Each Clifford gate admits a binary symplectic matrix representation acting on $[G_X \mid G_Z]$ through right multiplication. Importantly, entangling Clifford gates such as CZ do not correspond to simple permutations of the tripled binary representation. Consequently, permutation automorphism searches are structurally restricted to transformations generated by local Clifford operations together with qubit permutations.

The final symplectic transformation associated with a permutation automorphism may then be decomposed into elementary Clifford generators $\langle H, S, \mathrm{SWAP}\rangle$, following the procedure described in Ref. [7].

The logical action of the resulting circuit is determined by conjugating the logical operators with the reconstructed physical circuit and identifying the induced transformation on the encoded Pauli operators. In practice, this is computed using the tableau formalism implemented in `stim`.

C. Phase Recovery and Pauli Corrections

The binary and GF(4) representations used in automorphism searches ignore global Pauli phases. Consequently, automorphism packages such as `AutQEC` operate only on the Pauli support structure and cannot directly distinguish stabilizer generators differing by signs. This becomes particularly important for intermediate stabilizer codes arising during pieceable fault-tolerant constructions, where stabilizer generators may acquire nontrivial phases.

Suppose a Clifford circuit U maps stabilizer generators $\{g_i\}$ to

$$g_i' = U g_i U^\dagger.$$

The transformed generators may generate the same stabilizer group while differing from the original generators by signs and basis changes.

Ignoring phases, each original generator may be expressed as a product of transformed generators:

$$g_i \sim \prod_j (g_j')^{c_{ij}} = \tilde{g}_i.$$

Then $\tilde{g}_i = (-1)^{s_i} g_i$, with $s_i \in \{0, 1\}$.

To correct these sign mismatches, one seeks a Pauli operator P satisfying

$$P \tilde{g}_i P^\dagger = g_i.$$

Since Pauli conjugation flips the sign of a stabilizer generator precisely when the two operators anticommute, the correction problem reduces to solving a linear system over $\mathbb{F}_2$. Representing $g_i = (x_i | z_i)$ and $P = (p_X | p_Z)$, the commutation relation is determined by the symplectic inner product

$$\langle P, g_i \rangle = p_X \cdot z_i + p_Z \cdot x_i \pmod 2.$$

The required correction therefore satisfies

$$\langle P, \tilde{g}_i \rangle = s_i \pmod 2.$$

Stacking all generators gives the linear system $Ap = s$, which may be solved over $\mathbb{F}_2$. The resulting Pauli operator P restores the correct stabilizer phases after the automorphism transformation.

D. Embedded-Code Constructions

Since permutation automorphism searches are structurally restricted to local Clifford operations and qubit permutations, Ref. [7] introduced an embedding construction designed to generate entangling logical operations. Although highly creative, this approach is computationally expensive and appears difficult to scale to larger codes.

The construction embeds the original stabilizer code into a larger auxiliary code by introducing ancilla qubits. Each ancilla is associated with a selected pair of physical qubits on which one wishes to induce entangling operations. In the most general construction, up to

$$\binom{n}{2}$$

ancillas may be introduced, one for each possible pair of qubits.

The ancilla qubits are initialized in the $|0\rangle$ state. For each ancilla associated with a pair (i, j), CNOT gates are applied from the physical qubits i and j onto the ancilla. Automorphism searches are then performed on the enlarged embedded code. Certain permutation patterns involving the ancilla coordinates can subsequently be translated back into entangling operations acting on the original code qubits.

This translation relies on several circuit identities summarized in Fig. 3, adapted from Ref. [7].

The induced translations satisfy the following rules:

- Clifford operations acting only on original code qubits remain unchanged.
- An S gate acting on an ancilla qubit translates into a CZ gate between the qubits in the ancilla support.

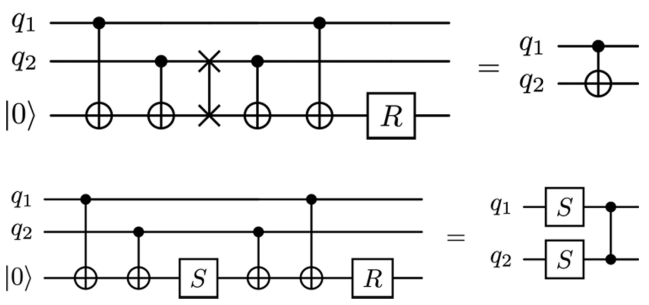

FIG. 3: Circuit identities used to translate automorphisms in the embedded code back into physical operations on the original code, adapted from Ref. [7].

- A SWAP operation between an ancilla and one of its support qubits translates into a CNOT operation between the corresponding support qubits.
- Hadamard gates acting on ancillas, as well as SWAP operations between ancillas, generally do not admit valid translations back into the original code space.

Additional details regarding the embedding construction and translation procedure may be found in the appendices of Ref. [7].

Although this embedding approach is conceptually interesting, it exhibits several important limitations. First, in our experiments it fails to identify useful entangling logical operators between multiple code blocks, including constructions resembling the round-robin pieceable fault-tolerant circuits discussed previously. Second, the search space grows extremely rapidly with the number of ancillas. For example, embedding interactions between two 5-qubit blocks may already require a substantially enlarged code, while embedding larger multi-block constructions can easily increase the number of coordinates to several dozens of qubits. Consequently, the resulting automorphism searches become computationally expensive and appear difficult to scale beyond relatively small examples.

IV. AUTOMORPHISMS OF INTERMEDIATE STABILIZER CODES

A key feature of pieceable fault-tolerant constructions is that the encoded code space evolves dynamically throughout the circuit. Between circuit pieces, the stabilizer generators change as entangling gates are applied, producing a sequence of intermediate codes with distinct symmetry structures. This naturally motivates studying the automorphisms of these intermediate stabilizer codes rather than only the automorphisms of the initial static code.

In this section, we focus on the logical $\overline{CZ}$ construction shown in Fig. 1, where all intermediate codes remain stabilizer codes.

We performed automorphism searches at multiple points throughout the circuit evolution. Initially, the two-block $[[10, 2, 3]]$ code possesses the automorphism structure $Aut(C_5^{\oplus 2}) \cong (Aut(C_5) \times Aut(C_5)) \rtimes S_2$, with $|Aut(C_5^{\oplus 2})| = 360^2 \cdot 2 = 259200.$, as explained in Sec.III A

After the prologue stage, the code retains essentially the same symmetry structure since the prologue consists only of local Clifford transformations, which preserve code equivalence. However, as physical CZ gates from the round-robin construction are introduced, the symmetry group rapidly decreases in size as the intermediate code becomes increasingly entangled. Empirically, the automorphism group order drops from roughly 10^3 elements to approximately 50, and eventually to only 8 elements after several intermediate CZ operations. As the circuit approaches the epilogue stage and some entanglement structure is effectively unraveled, the automorphism group begins increasing again.

The automorphism searches were performed using the `AutQEC` Python package together with the underlying `MAGMA` algebra system.

Although the resulting automorphisms are still generated by local Clifford transformations and coordinate permutations, performing the search directly on intermediate stabilizer codes produces useful fused logical constructions. Rather than obtaining independent logical Clifford operators followed by the round-robin circuit, the resulting constructions naturally combine the logical operations with the entangling structure already present in the intermediate code.

Schematically, many of the resulting circuits take the form

$$P_0' P_1' \, \overline{CZ}_{01} \, P_0 P_1,$$

where the P_i are logical single-qubit Clifford operators. These fused constructions are often cheaper than independently implementing each logical Clifford together with the logical entangling gate. Such fused primitives may be useful for compiler optimizations, logical circuit synthesis, and reducing the overhead of encoded Clifford layers.

Many of these fused constructions could equivalently be interpreted as logical Clifford automorphisms of the original code combined with the existing round-robin CZ structure. However, performing the search directly on intermediate codes provides a natural mechanism for discovering such optimized constructions automatically.

One particularly interesting example is a fused logical $\overline{CZ}$-$\overline{CNOT}$ construction requiring the same number of physical CZ gates as the original logical $\overline{CZ}$.

This structure can be understood by commuting the Hadamard gate through the first $CNOT$, thereby converting the interaction into a CZ-type entangling structure, as illustrated in Fig. 5.

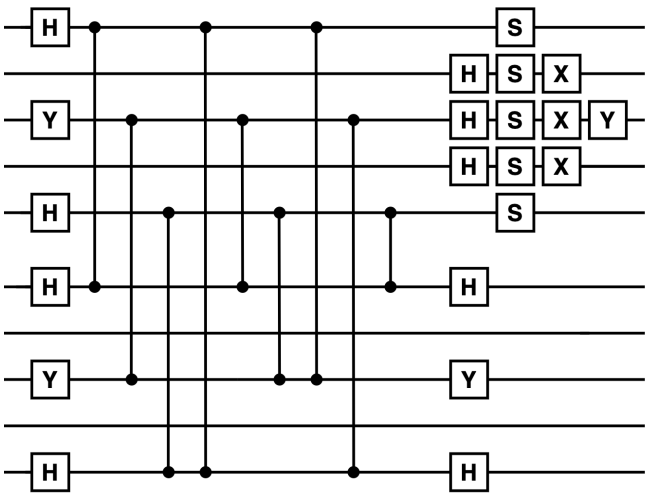

FIG. 4: Example of a fused logical construction obtained from automorphisms of an intermediate stabilizer code.

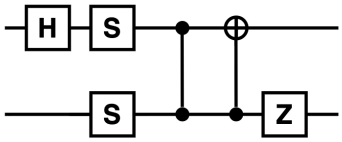

FIG. 5: Logical action of the fused intermediate-code construction.

V. NON-STABILIZER INTERMEDIATE CODES AND CWS REPRESENTATIONS

In the previous sections, we studied automorphisms of dynamically evolving intermediate stabilizer codes arising during pieceable fault-tolerant constructions. Although these searches produced useful fused logical Clifford constructions, the resulting transformations remained fundamentally generated by local Clifford operations and coordinate permutations. Consequently, the induced circuits could still largely be interpreted as logical Clifford automorphisms combined with the original round-robin entangling structure.

To move beyond this restriction, we now consider pieceable constructions of logical gates $\overline{C^hZ}$ for $h \geq 2$, such as the logical $\overline{CCZ}$ construction shown previously in Fig. 2. Unlike the $\overline{CZ}$ case, these constructions contain non-Clifford physical gates during the round-robin stage. As a result, the intermediate codes are generally no longer stabilizer codes.

This observation naturally motivates extending the automorphism search framework beyond additive stabilizer codes into the more general CWS formalism introduced in Sec. II E. Since the intermediate circuits already contain non-Clifford entangling structure, the resulting automorphisms are no longer cleanly separable into independent local Clifford layers together with the original round-robin construction. Instead, the local Clifford structure and the non-Clifford entangling structure remain intrinsically intertwined throughout the intermediate evolution.

In this section, we propose a framework for studying automorphisms of such non-stabilizer intermediate codes through their equivalent CWS representations. A CWS code is represented by a pair

$$(\mathcal{G}, \mathcal{C}),$$

where $\mathcal{G}$ denotes the graph-state structure and $\mathcal{C}$ denotes the associated classical binary code. The construction applies generally to pieceable logical $\overline{C^hZ}$ gates between arbitrary stabilizer code blocks. For concreteness, we illustrate the procedure using the logical $\overline{CCZ}$ construction between three blocks of the $[[5, 1, 3]]$ code shown previously in Fig. 2.

A key observation is that C^hZ gates act diagonally in the computational basis and therefore preserve Pauli operators containing only Z components. This allows the intermediate non-stabilizer structure to retain a tractable description within the CWS framework.

We begin with three copies of the $[[5, 1, 3]]$ code, forming a combined $[[15, 3, 3]]$ stabilizer code and apply the prologue circuit of the pieceable construction. The next step is to convert this stabilizer representation into the standard CWS graph-state form. Recall that any stabilizer state is locally Clifford equivalent to a graph state whose generators take the form

$$K_i = X_i Z^{A_i}$$

where A is the adjacency matrix of the graph $\mathcal{G}$.

To obtain this representation, we apply Gaussian elimination on the binary symplectic representation together with an automatically determined layer of local Hadamard transformations chosen so that the X block becomes invertible. The resulting adjacency matrix A defines the graph-state component $\mathcal{G}$ of the CWS representation.

Algorithmically, this procedure consists of:

1. converting stabilizers into binary symplectic form
2. applying local Hadamard layers to make the X block invertible
3. performing Gaussian elimination over $\mathbb{F}_2$
4. extracting the graph adjacency matrix A
5. reconstructing graph-state stabilizers $K_i = X_i Z^{A_i}$

The logical operators are then mapped into CWS codewords defining the classical code $\mathcal{C}$. For graph-state stabilizers $K_i = X_i Z^{A_i}$, a general Pauli operator $X^x Z^z$ is equivalent, up to stabilizer multiplication, to a purely Z-type operator

$$Z^{z+xA}$$

Consequently, the logical operators may be represented as classical binary codewords within $\mathcal{C}$. Applying this procedure to the post-prologue $[[15, 3, 3]]$ code yields the graph-state stabilizers

$$
\begin{array}{c}
XZIZZIIIIIIIIII \\
ZXIZIIIIIIIIIII \\
IIXZZIIIIIIIIII \\
ZZZXIIIIIIIIIII \\
ZIZIXIIIIIIIIII
\end{array}
$$

$$
\begin{array}{c}
IIIIIXZIZZIIIII \\
IIIIIZXIZIIIIII \\
IIIIIIIXZZIIIII \\
IIIIIZZZXIIIIII \\
IIIIIZIZIXIIIII
\end{array}
$$

$$
\begin{array}{c}
IIIIIIIIIIXZIZZ \\
IIIIIIIIIIZXIZI \\
IIIIIIIIIIIIXZZ \\
IIIIIIIIIIZZZXI \\
IIIIIIIIIIZIZIX
\end{array}
$$

The resulting classical code $\mathcal{C}$ is generated by the codewords

$$
\begin{array}{c}
000000000000000 \\
000000000001001 \\
010010000000000 \\
010010000001001 \\
000000100100000 \\
000000100101001 \\
010010100100000 \\
010010100101001
\end{array}
$$

The central observation appears once a physical CCZ gate from one of the round-robin circuit pieces is applied. The classical code $\mathcal{C}$ remains unchanged throughout the evolution. Since CCZ gates are diagonal in the computational basis, they preserve the classical binary codewords associated with the Z-type word operators. The non-Clifford dynamics are therefore transferred entirely into the graph component $\mathcal{G}$.

Applying a CCZ gate generally introduces higher-order phase interactions that cannot be represented by ordinary graph edges alone. Consequently, the graph component evolves into a hypergraph containing higher-order hyperedges.

This produces a particularly interesting structure: automorphism searches now correspond to transformations preserving both the classical code $\mathcal{C}$ and the hypergraph structure $\mathcal{G}$. We therefore define the intermediate CWS automorphism group as

$$Aut(\mathcal{C}, \mathcal{G}) = \{\pi \in S_n : \pi(\mathcal{C}) = \mathcal{C},\ \pi(\mathcal{G}) = \mathcal{G}\}.$$

An important observation is that the classical code $\mathcal{C}$ remains unchanged throughout the entire pieceable CCZ circuit evolution, since all intermediate operations are diagonal controlled-phase gates. Therefore, the automorphism search on $\mathcal{C}$ only needs to be performed once, after which one may test which transformations preserve the evolving hypergraph structure $\mathcal{G}$ at different circuit stages. This framework naturally allows for constructions containing mixed higher-order controlled-phase interactions, including both CZ and CCZ operations.

We performed the automorphism search using `MAGMA`. Interestingly, the classical code possesses a very large automorphism group, $|Aut(\mathcal{C})| = 17418240$, generated by 13 independent generators. However, none of the nontrivial automorphisms preserve the intermediate hypergraph structure.

This suggests that the dominant symmetry-breaking mechanism arises from the hypergraph structure $\mathcal{G}$ rather than from the classical code $\mathcal{C}$. In the current formulation, the automorphism search acts primarily through coordinate permutations on the classical binary code, while preserving the hypergraph imposes an extremely restrictive constraint.

These observations motivate several possible extensions of the framework. One possible direction is constructing enlarged embeddings of both the classical code $\mathcal{C}$ and hypergraph structure $\mathcal{G}$ analogous to the tripled symplectic embedding used previously for stabilizer automorphism searches. Such constructions could potentially allow permutation searches to induce additional gate types beyond SWAP operations, including local Clifford operations such as H and S.

Another possibility is incorporating diagonal Clifford operations such as CZ directly into the embedding procedure, since these operations preserve the classical code structure. More generally, one may consider extending the embedding framework itself by introducing auxiliary hypergraph ancilla nodes analogous to the ancilla constructions used in the stabilizer embedding approach.

The main challenge is that the hypergraph-preservation condition is extremely rigid. Relaxing this requirement from exact preservation of $\mathcal{G}$ to preservation up to physically equivalent CWS representations may substantially enlarge the accessible symmetry space.

Due to the limited scope and timescale of this project, we leave the development of such constructions as future work. Several major open problems remain:

1. Constructing embeddings of both the classical code $\mathcal{C}$ and hypergraph structure $\mathcal{G}$ such that permutation automorphisms induce gate sets beyond SWAP operations.

2. Relaxing exact hypergraph preservation to equivalence under local Clifford transformations or equivalent CWS representations.

3. Developing a systematic pipeline translating automorphisms of intermediate CWS representations back into physical logical circuits acting on the original stabilizer codes.

VI. CONCLUSION

In this work, we investigated automorphisms of dynamically evolving intermediate codes arising during pieceable fault-tolerant logical gate constructions. Rather than studying only automorphisms of static stabilizer-code representations, we analyzed the sequence of intermediate codes generated throughout the evolution of pieceable logical circuits. This viewpoint naturally treats pieceable fault-tolerant constructions as dynamically evolving families of quantum codes possessing distinct symmetry structures at different stages of the circuit evolution.

For pieceable logical $\overline{CZ}$ constructions on the $[[5, 1, 3]]$ code, the intermediate codes remain stabilizer codes throughout the evolution, allowing automorphism searches to be performed directly on the evolving intermediate stabilizer representations. Although the resulting transformations remained generated by local Clifford operations and coordinate permutations, the searches produced useful fused logical constructions combining logical Clifford operations with the entangling structure already present in the circuit. We additionally observed that the automorphism groups of the intermediate codes shrink rapidly as the round-robin entangling structure becomes increasingly complex.

We then extended the framework to pieceable logical $\overline{CCZ}$ constructions, where the intermediate codes become non-stabilizer due to the presence of non-Clifford gates. Using equivalent Codeword Stabilized (CWS) representations, we showed that the associated classical binary code remains invariant throughout the intermediate CCZ evolution, while the non-Clifford dynamics are transferred into the evolving graph and hypergraph structure. Although the resulting classical code possesses a large automorphism group, preserving the hypergraph structure strongly suppresses nontrivial automorphisms.

Overall, these results illustrate both the potential and the limitations of permutation-based symmetry methods for fault-tolerant logical gate synthesis. More broadly, they suggest that dynamically evolving intermediate-code symmetries may provide a useful framework for studying optimized fault-tolerant logical constructions beyond static stabilizer-code automorphisms.

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press (2010).

[2] B. Eastin and E. Knill, "Restrictions on Transversal Encoded Quantum Gate Sets," *Phys. Rev. Lett.* **102**, 110502 (2009).

[3] S. Bravyi and R. König, "Classification of Topologically Protected Gates for Local Stabilizer Codes," *Phys. Rev. Lett.* **110**, 170503 (2013).

[4] S. Bravyi and A. Kitaev, "Universal Quantum Computation with Ideal Clifford Gates and Noisy Ancillas," *Phys. Rev. A* **71**, 022316 (2005).

[5] T. J. Yoder, R. Takagi, and I. L. Chuang, "Universal Fault-Tolerant Gates on Concatenated Stabilizer Codes," *Phys. Rev. X* **6**, 031039 (2016).

[6] T. Jochym-O'Connor and R. Laflamme, "Using Concatenated Quantum Codes for Universal Fault-Tolerant Quantum Gates," *Phys. Rev. Lett.* **112**, 010505 (2014).

[7] M. Grassl, M. Roetteler, and T. Beth, "On Optimal Quantum Codes," *Int. J. Quantum Inf.* **2**(1), 55–64 (2004).

[8] M. Grassl, M. Roetteler, and T. Beth, "Fault-Tolerant Logical Clifford Gates from Code Automorphisms," arXiv:quant-ph/0302129 (2003).

[9] A. R. Calderbank, E. M. Rains, P. W. Shor, and N. J. A. Sloane, "Quantum Error Correction via Codes Over GF(4)," arXiv:quant-ph/9608006 (1996).

[10] A. Cross, G. Smith, J. A. Smolin, and B. Zeng, "Codeword Stabilized Quantum Codes," *IEEE Transactions on Information Theory* **55**(1), 433–438 (2009).

[11] D. Gottesman, "The Heisenberg Representation of Quantum Computers," *Proceedings of the XXII International Colloquium on Group Theoretical Methods in Physics* (1998), arXiv:quant-ph/9807006.

[12] A. Paetznick and B. W. Reichardt, "Universal Fault-Tolerant Quantum Computation with Only Transversal Gates and Error Correction," *Phys. Rev. Lett.* **111**, 090505 (2013).

[13] D. Litinski, "A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery," *Quantum* **3**, 128 (2019).